

Agentic AI System for Enterprise Cloud Migration

Timothy Wong • Alessandro Abati

London Stock Exchange Group (LSEG)

7 Oct 2026

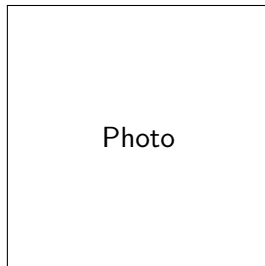


About the Speakers



Timothy Wong

Tim is an AI/ML engineer at LSEG with over ten years experience in enterprise data science applications. He hosted talks at EARL 2016, 17, 19 & 24.



Alessandro Abati

placeholder



LSEG

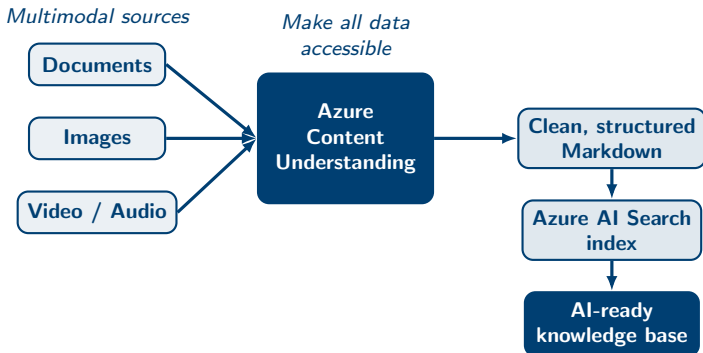
LSEG is a leading financial markets infrastructure and data provider operating connected businesses to serve customers across the entire financial markets value chain.

- ▶ **London Stock Exchange**
- ▶ **FTSE Russell:** Indices across asset classes, including FTSE 100, FTSE 250, Russell 2000, and ESG indices.
- ▶ **Data & Analytics:** Workspace platform, pricing, reference data.
- ▶ **Clearing:** LCH, multi-asset class clearing house.
- ▶ **Scale:** ~26 000 employees across many countries.

1. **Business Requirements and Context**
2. **Agentic Technical Deep Dive**
 - 2.1 **Data Preprocessing**
 - 2.2 **Inside Agentic Retrieval**
 - 2.3 **Agentic Orchestration**
 - 2.4 **Agents' Memory**
 - 2.5 **Agents' Tools**
 - 2.6 **E2E Orchestration Phases**
3. **Infrastructure & Backend**

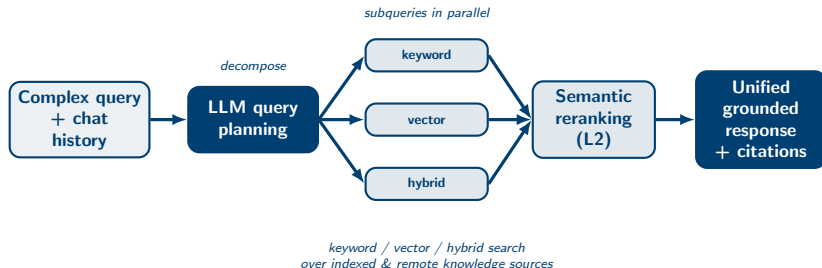
- ▶ **Problem:** *placeholder* — what makes enterprise cloud migration slow, risky and manual.
- ▶ **Goal:** *placeholder* — automate discovery, analysis and recommendation with AI agents.
- ▶ **Stakeholders & scope:** *placeholder*.
- ▶ **Success criteria:** *placeholder*.

Data Preprocessing - Making Documentation AI-Ready



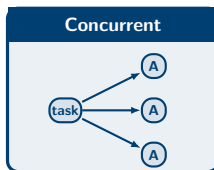
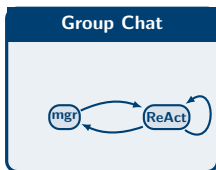
- **Content Understanding:** one analyzer for *any* modality - OCR, layout, tables and figure descriptions turn messy files into clean Markdown.
- **Make all data accessible:** charts, diagrams and scans become searchable text, so nothing is lost to downstream agents.
- **Azure AI Search:** content is chunked and indexed into an AI-ready knowledge base for agentic retrieval.

Inside Agentic Retrieval - A Multi-Query Pipeline



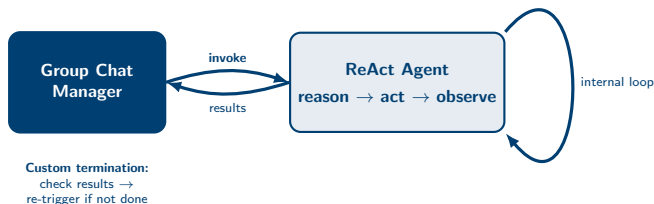
- ▶ **Beyond classic RAG:** handles multi-part questions, chat context, typos and paraphrasing that a single query can't.
- ▶ **Query planning:** an LLM breaks the query down into focused subqueries that run *in parallel*, each semantically reranked.
- ▶ **Grounded output:** best results are merged into one response with source citations and an activity log for agents.

Agentic Orchestration - Overview



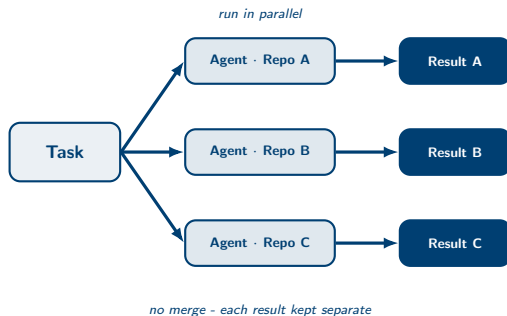
- ▶ **Framework:** Microsoft **Semantic Kernel** agent orchestration, tailored to our use case.
- ▶ **Two patterns:** *Group Chat* (1 manager + 1 ReAct agent, custom termination) and *Concurrent* (parallel, per-repo agents).
- ▶ **Writer/Reviewer** is a *Round-Robin* flavour of Group Chat - same orchestration, alternating turns.

Agentic Orchestration - Group Chat



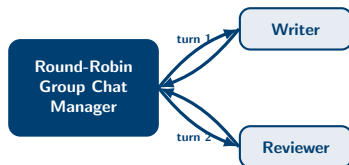
- ▶ **Our setup:** *one* manager coordinating a *single* ReAct agent - not a multi-agent debate.
- ▶ **Why a manager:** a ReAct agent's reason-act loop is trained to stop early; the manager inspects its results and *re-triggers* the loop when the task isn't complete.
- ▶ **Net effect:** a custom termination strategy on top of the agent's own reason-act loop.

Agentic Orchestration - Concurrent



- **Our setup:** the task fans out to one agent *per repo*, all running in parallel.
- **No aggregation:** results are *not* combined - each agent only needs its own repo's context.
- **Benefits:** parallelism for speed, plus clean isolation - no cross-repo context bleed.

Agentic Orchestration - Writer/Reviewer



What Round-Robin overrides

SelectNextAgent - picks the next agent by a fixed **cyclic order** (a round-robin cursor).

Everything else is inherited from the base GroupChatManager.

- ▶ **Same pattern:** reuses the *Group Chat* orchestration with the RoundRobinGroupChatManager flavour - no custom manager logic needed.
- ▶ **Deterministic turns:** Writer drafts, Reviewer critiques, alternating for a fixed number of rounds (writer gets the last word).
- ▶ **Benefits:** a built-in quality gate - iterative critique lifts output quality with predictable turns and no custom manager logic.

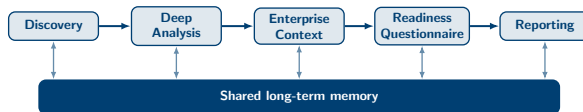
Agents' Memory

Shared long-term memory

- ▶ A single shared memory accumulates findings across the whole run.
- ▶ Each phase reads what earlier phases discovered and writes back its own results.
- ▶ Later phases build on earlier work instead of re-reading the raw inputs.

Passing context between phases

- ▶ The pipeline is serial: every phase produces a report.
- ▶ Each report, with the shared memory, becomes the input context for the next phase.
- ▶ By the final phase, agents reason over the full accumulated picture.



Reports flow forward; every phase reads from and writes to the shared memory.

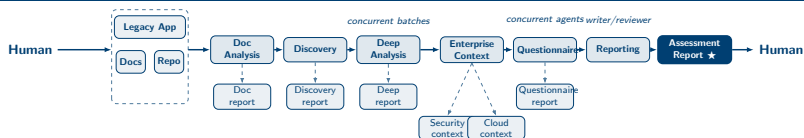
What tools are available

- ▶ **File access:** browse and read the codebase and documents.
- ▶ **Knowledge retrieval:** ask questions over indexed documents; answers cite their sources.
- ▶ **Enterprise knowledge:** search organisational guidance (security, cloud, DevOps standards).
- ▶ **Diagrams:** create, validate and fix architecture diagrams, rendered offline.

How agents use them

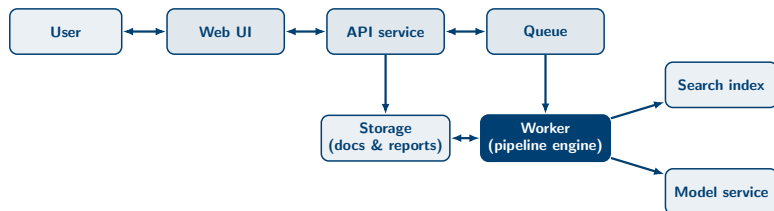
- ▶ Each agent is given only the tools relevant to its task.
- ▶ Tools turn raw inputs into structured, grounded findings.
- ▶ Grounded retrieval keeps answers anchored to real evidence.
- ▶ Tool use is logged for observability and cost tracking.

E2E Orchestration Phases



Inputs (documents & code) enter on the left; each phase records its findings and passes context forward.

- ▶ **Discovery:** group the code and documents into logical components. *(Output: a map of the system.)*
- ▶ **Deep Analysis:** assess each component's migration readiness, in parallel. *(Output: per-component findings.)*
- ▶ **Enterprise Context:** apply organisational security and cloud standards. *(Output: policy-aligned recommendations.)*
- ▶ **Readiness Questionnaire:** answer a standard cloud-readiness questionnaire. *(Output: structured answers.)*
- ▶ **Reporting:** consolidate and review all findings. *(Output: the final assessment report.)*



- ▶ **Flow:** users upload through the web UI; the API stores inputs and queues a job; a worker then runs the phase pipeline.
- ▶ **Shared services:** the worker draws on storage, the search index and the model service to complete each phase.
- ▶ **Isolated environment:** the system runs fully offline, with automatic retries on transient failures.

Thank you
Any questions?